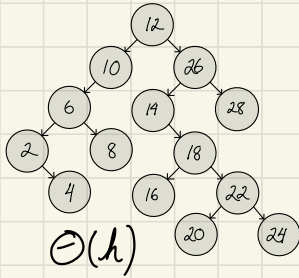



Binary Search Trees



- Let T be a binary tree. T is a BST if for each node n in T :

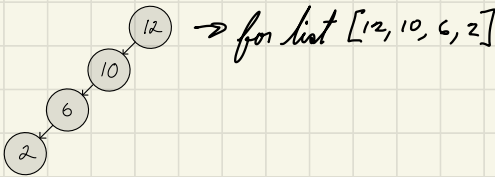
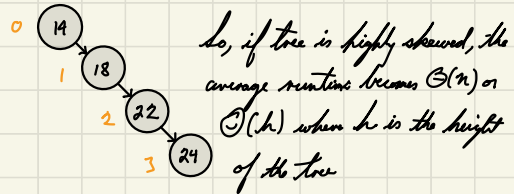
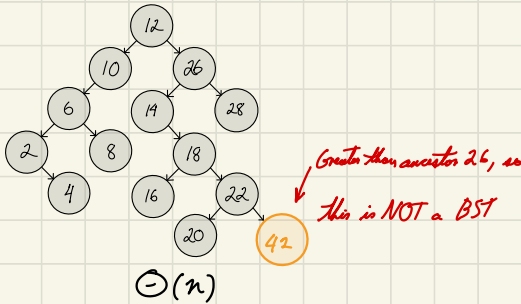
- All keys stored in the left subtree of n are less than the key in n
- $\checkmark \checkmark \checkmark \checkmark \checkmark$ right $\checkmark \checkmark \checkmark \checkmark$ greater than $\checkmark \checkmark \checkmark$

bst = Binary Search Tree()

lst = [14, 18, 22, 24]

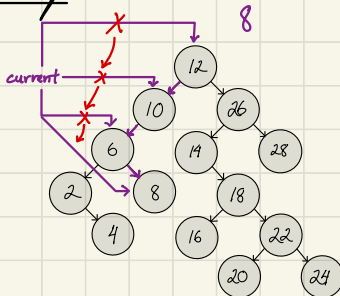
for num in lst:

bst.insert(num)



Operations

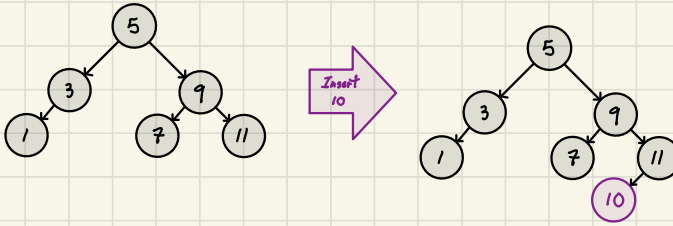
• Lookup:



1. If current node is None, the value isn't in BST.
2. If $8 == \text{node.data}$, return node.data.
3. If $8 < \text{node.data}$, current = node.left of search
4. $\checkmark 8 > \text{node.data}$, $\checkmark \checkmark \checkmark \checkmark$.right $\checkmark \checkmark$

- $O(1)$ if root.data == target
- $O(h)$ in general (h. of skewed trees)
 - ↳ $(\log n)$ for balanced trees
- $O(h)$ for skewed trees

• Insertion:



1. If tree is empty, the new node becomes the root ^(v)
2. Otherwise, starting at the root:
 - If $v < \text{current node.data}$
 - If there's a left child, "recurse" into left child
 - If there isn't v becomes the left child
 - If $v > \text{current node.data}$
 - If there's a right child, "recurse" into right child
 - If there isn't v becomes the right child
3. If $v == \text{node.data}$, ignore. BSTs don't allow duplicates by default.

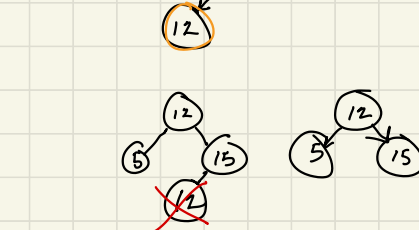
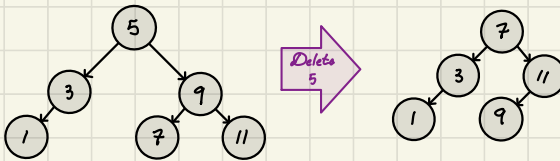
$\Omega(1)$: Tree is empty

$\Theta(\log n)$: Balanced tree

$\Theta(n)$: Generally

$\mathcal{O}(h)$: skewed tree

• Deletion:



1. Search for target node
2. Depending on the children situation
 - No children: Disconnect target
 - One child: Promote child to the target node
 - Two children:
 1. Find in-order successor (smallest)
 2. Replace target's value w/ successor's
 3. Recursively delete successor

$\Omega/\Theta(\log n)$

$\mathcal{O}(h)$